esprit Ecole Supérieure Privée d'Ingénierie et de Technologies	**Examen** Semestre : 1 ☐ 2 ■

Module : **Techniques de Conteneurisation** Enseignant(s) : **UP Système** Classe(s) : 4Arctic, 4Iosys & 4Sleam Documents autorisés : OUI ☐ NON ■ Date : **03/07/2025** Heure : 8H30	Session**: Rattrapage** Nombre de pages : 7 Durée : 1h30
N° Carte : ……..... Nom et Prénom : …….	Classe : ……..

Exercice 1 : Questions de réflexion (4pts)

1. Quelle commande permet de créer une image Docker nommée **myimg** à partir de l'état actuel du conteneur nommé **my_container** ? (0,5pt)

docker commit my_container myimg

2. On considère cet extrait du Dockerfile : (1 pt)

```
ENTRYPOINT ["echo", "Hello"]
CMD ["World"]
```

Sachant que « **image-name** » est le nom de l'image créée à partir de ce Dockerfile, qu'est-ce qu'on obtient si on exécute :

a. docker run image-name

Hello World **(0,5 pt)**

b. docker run image-name Docker

Hello Docker **(0,5 pt)**

3. Quelle approche utilise ce Dockerfile et quels sont ses avantages ? (1pt)

```
FROM node:18 as builder
WORKDIR /app
COPY . .
RUN npm install && npm run build

FROM nginx
COPY --from=builder /app/build /usr/share/nginx/html
```

C'est une approche **multi-stage build**. **(0,5 pt)**

Avantages : **(0,5 pt)**

- Réduit la taille de l'image finale.
- Sépare l'environnement de build de celui de production.
- Ne garde que les fichiers nécessaires à l'exécution.

4. En quoi consiste le principe de **COW** dans Docker et quels sont ses avantages ? (1 pt)

Lorsqu'un conteneur est créé à partir d'une image existante, il partage l'image en **lecture seule**. Docker crée une **nouvelle couche d'écriture** contenant uniquement les différences entre l'image de base et le conteneur. **(0,5 pt)**

Avantages : **(0,5 pt)**

- Gain d'espace
- Création rapide des conteneurs
- Isolation des changements

5. Quelle est la principale différence entre les instructions **RUN** et **CMD** dans un Dockerfile ? (0,5pt)

RUN est une instruction utilisée lors de la construction de l'image. Elle sert à exécuter des commandes comme l'installation de paquets. Son effet est enregistré dans l'image finale. **(0,25 pt)**

CMD est une instruction utilisée lors de l'exécution du conteneur. Elle définit la commande par défaut à exécuter lorsque le conteneur démarre. **(0,25 pt)**

Exercice 2 : (6 pts)

Un ingénieur DevOps vient de rejoindre l'équipe technique d'un site de e-commerce. Sa mission consiste à mettre en place une version de test de l'application en environnement local à l'aide de Docker.

L'objectif est de déployer les différents composants de l'application dans des conteneurs Docker, selon l'architecture suivante :

Frontend

- Service basé sur l'image publique **node:18-alpine**.
- Accessible sur le port **3000**.

Backend

- Développé en **Python Flask**.
- À construire à partir d'un **Dockerfile** situé dans le répertoire **./backend.**
- Exposé sur le port **5000**.
- Dépend de la base de données pour fonctionner correctement.

Base de données MySQL

- Version : **MySQL 5.7**.
- Nom du conteneur : **mysql-db**.
- Nom de la base de données : **ecommerce**.

- Identifiants de connexion :
    - Utilisateur : devuser
    - Mot de passe : devpass

Tous les services doivent communiquer via un réseau Docker privé nommé **ecom-net**.

1. Quel outil Docker permet d'orchestrer ces services sur une seule machine ? (0.25 pt)

Docker Compose

2. Dans quel fichier décrit-on l'architecture complète des services ? (0.25 pt)

docker-compose.yml

3. Soit le fichier suivant, permettant de décrire l'architecture de cette pile d'applications. En se basant sur les informations mentionnées précédemment, Compléter les champs manquants. (2 pts) **(0,25 pt pour chaque réponse)**

```
version: "3.8"
services:
  frontend:
    image: node:18-alpine (1)
    ports:
      - "3000:3000" (2)
    working_dir: /app
    volumes:
      - ./frontend:/app
    command: npm start
    networks:
      - ecom-net

  backend:
    build: ./backend (3)
    ports:
      - "5000:5000" (4)
    networks:
      - ecom-net
    depends_on:
      - db (5)
  db:
    image: mysql:5.7
    container_name: mysql-db (6)
    environment:  (7)
      MYSQL_DATABASE: ecommerce
      MYSQL_USER: devuser
      MYSQL_PASSWORD: devpass
      MYSQL_ROOT_PASSWORD: rootpass
    networks:
      - ecom-net (8)

networks:
  ecom-net:
```

Figure 1

4. Quelle commande permet de démarrer cette pile d'applications en utilisant le fichier de la **Figure 1** ? (0,5 pt)

docker-compose up -d

5. L'application e-commerce précédemment déployée en local doit maintenant être mise en production.
   a. Proposer une solution Docker permettant d'étendre le déploiement de l'application sur plusieurs serveurs afin de l'adapter à un environnement de production. (0,25pt)

Docker Swarm

   b. Quelle commande faut-il exécuter pour initialiser cet environnement de production ? (0.25 pt)

docker swarm init

   c. Indiquer la commande permettant de déployer l'application complète sous le nom **ecom-app** dans l'environnement de production, en utilisant le fichier présenté à la **Figure 1.** (0,75 pt)

docker stack deploy -c docker-compose.yml ecom-app

6. La capture ci-dessous montre l'état actuel des services déployés dans l'environnement de production.

ID	NAME	MODE	REPLICAS	IMAGE
a8z2m9kqmx	ecom-app_frontend	replicated	2/2	node:18-alpine
y1x9u7xql3	ecom-app_backend	replicated	1/1	backend:latest
k2f93kgqlx	ecom-app_db	replicated	1/1	mysql:5.7

   a. Combien d'instances du frontend tournent actuellement ? (0,25pt)

2 instances

   b. Quelle commande permet la mise à l'échelle du service frontend à 4 réplicas ? (0.5 pt)

docker service update --replicas 4 ecom-app_frontend

7. Le résultat de l'exécution de la commande « docker node ls » est présenté dans la capture ci-dessous :

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSIO
k4t9dcvqqqq3u3gsvab2pz4hj *	node1	Ready	Active	Leader	24.0.5
tq7fdqv9z8rjv9c3pqz0p1xju	node2	Ready	Active	Reachable	24.0.5
gl28dyk98d7sdqkch0q1zj84h	node3	Ready	Active		24.0.5

a. Quel type de réseau permet la communication entre les conteneurs répartis sur les différents nœuds de l'environnement de production ? (0,25 pt)

overlay

b. Quelle commande permet de convertir le nœud **node3** en manager ? (0,5 pt)

docker node promote node3

c. Quelle commande permettrait de retirer **node2** du cluster proprement ? (0,25 pt)

docker node rm node2

Exercice 3 : Problème (10pts)

Un stagiaire au sein d'une équipe DevOps a pour mission de **conteneuriser une application Python**. Cette application est simple : elle affiche un message et écrit une ligne dans un fichier de log à chaque exécution.

1. Compléter le fichier Dockerfile ci-dessous. (2 pts) **(0,25 pt pour chaque réponse)**

```
# Image de base
FROM python:3.11-slim
# Métadonnées
LABEL version="1.0"
# Définir une variable d'environnement
ENV APP_ENV=production
# Définir le répertoire de travail
WORKDIR /app
# Copier les fichiers du contexte vers le conteneur
COPY . .
# Installer les dépendances
RUN pip install -r requirements.txt
# Exposer un port
EXPOSE 8000
# Définir la commande par défaut à exécuter
CMD ["python", "app.py"]
```

Dockerfile

2. Indiquer les commandes nécessaires pour :

a. Construire une image Docker nommée **python-app** à partir du Dockerfile ci-dessus. (0,25 pt)

docker build -t python-app .

b. Exécuter un conteneur nommé **mon-app** à partir de cette image. (0,5 pt)

docker run --name mon-app python-app

c. Afficher le contenu du fichier "requirements.txt" à partir du conteneur **mon-app** (0,75 pt)

docker exec mon-app cat requirements.txt

4. L'application écrit un message dans un fichier « output.txt » situé dans le dossier ***/app*** à chaque exécution. Cependant, après avoir supprimé puis relancé le conteneur créé à partir de la même image, le fichier est réinitialisé et les messages précédents disparaissent.

   Expliquer pourquoi les données contenues dans ***/app/output.txt*** sont perdues après suppression du conteneur. (0,5 pt)

   Lorsque vous créez et lancez un conteneur, Docker crée une couche de stockage temporaire où toutes les modifications sont enregistrées. Mais dès que le conteneur est supprimé, cette couche est détruite donc le système de fichiers d'un conteneur Docker est éphémère

5. Afin de résoudre ce problème, proposer deux solutions possibles en complétant le tableau ci-dessous. Cocher la case correspondante à chaque critère pour chacune des solutions. (1,75 pts) **(0,25 pt pour chaque réponse)**

	Solution 1 : Volumes	**Solution 2 :** Bind mounts
Géré automatiquement par Docker	X	
Géré manuellement par l'utilisateur		X
Stocké dans /var/lib/docker/volumes/, caché du système hôte.	X	
Directement lié au système de fichiers de l'hôte		X
Complètement isolé du système de fichiers de l'hôte	X	

6. L'application est désormais en phase de **déploiement sur plusieurs environnements** (développement, pré-production, production). Dans ce contexte multi-environnement, quelle solution parmi celles proposées précédemment est la plus adaptée ? Donner

également les commandes appropriées à utiliser, pour lancer le conteneur **mon-app** en utilisant cette solution. (1,5pts)

On doit utiliser les volumes nommés **(0,25 pt)**
docker volume create app_data **(0,5 pt)**
docker run –name mon-app -v app_data:/data python-app **(0,75 pt)**

7. On souhaite maintenant lancer un deuxième conteneur **logs**, basé sur une image **ubuntu**, afin d'inspecter ou de lire les logs de l'application **python-app** sans modifier le code source ni reconstruire l'image. Donner la commande appropriée pour réaliser cette opération. (1pt)

   docker run -it --name logs -v app_data:/data Ubuntu

8. L'équipe cherche à comprendre et à optimiser la configuration réseau des conteneurs dans le cadre d'un déploiement **multi-conteneurs sécurisé**.
   Par défaut, Docker utilise le mode réseau **bridge** pour permettre la communication entre les conteneurs.
   Expliquer comment le mode réseau **bridge** se distingue des modes **host** et **none**, en termes de sécurité et d'isolation ? (0,75pt)
   - Le mode réseau **bridge** isole les conteneurs dans un réseau privé interne. Il offre une bonne sécurité et une isolation correcte. **(0,25 pt)**
   - En mode **host**, le conteneur partage directement le réseau de l'hôte. Cela réduit l'isolation et rend le conteneur moins sécurisé. **(0,25 pt)**
   - En mode **none**, le conteneur n'a aucun accès réseau. Il est totalement isolé du point de vue réseau, ce qui le rend très sécurisé. **(0,25 pt)**

9. Afin d'isoler les communications réseau de l'application des autres conteneurs Docker tournant sur la même machine:

   a. Créer un réseau Docker personnalisé de type bridge nommé **app-network**. (0,25pt)

      docker network create --driver bridge app-network

   b. Lancer ensuite le conteneur **mon-app**, en le connectant à ce réseau **app-network**, et exposer le port 8000 du conteneur vers le port 8080 de l'hôte. (0,75pt)

      docker run -d --name mon-app --network app-network -p 8080:8000 python-app